SUMMARIZER

06 02 2022

ACADEMIA DE CÓDIGO @LISBOA @B.A
João Carneiro

- ABSTRACT
  - INTERFACES
- MISCELLANEOUS

Abstract

- Abstract is a type of <Class>
- Abstract keeps your code safe.
- Abstract can work as a guideline to help you through
- Abstract can have abstract and non abstract methods
- ORGANIZATION.
- SINGLE PARENT!

public abstract name {}

can NOT be instantiated.

it can have final methods

Abstract method
(ex. speak)

Non abstract method
(ex. output its name)

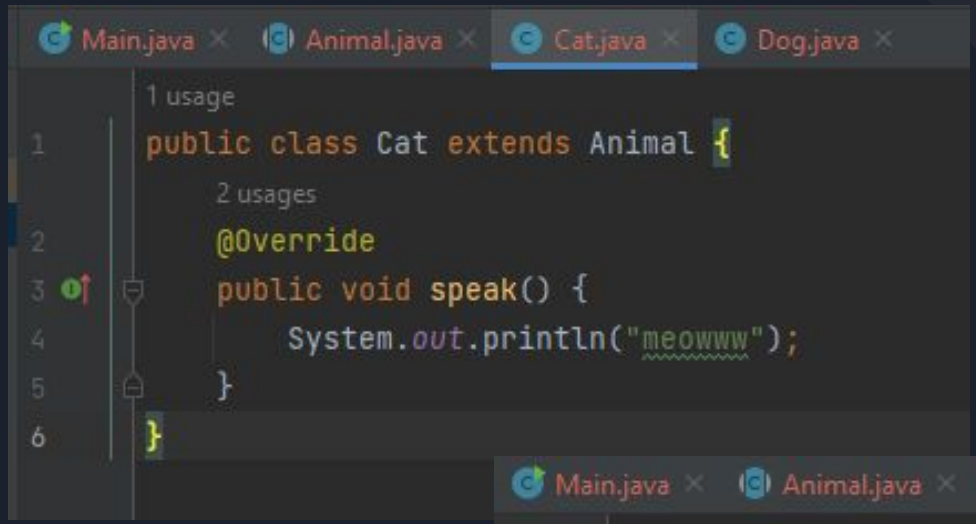
It can have constructors

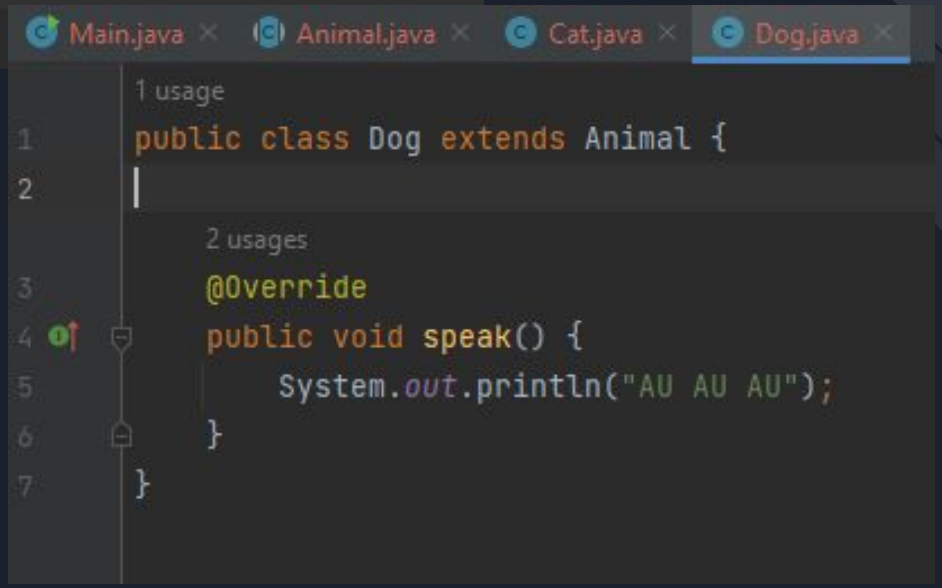
Abstract keeps you focused on **WHAT** an object does, instead of **HOW** it does it.

```java
4 usages   2 inheritors
public abstract class Animal {
    3 usages
    int weight;
    5 usages
    String name;

    2 usages   2 implementations
    public abstract void speak();

    no usages
    public void getName() {
        System.out.println("My name is :" + name);
    }
}
```


Main.java
Animal.java
Cat.java
Dog.java
1 usage
public class Cat extends Animal {
2 usages
@Override
public void speak() {
System.out.println("meowww");
}
}


Main.java
Animal.java
Cat.java
Dog.java
1 usage
public class Dog extends Animal {
2 usages
@Override
public void speak() {
System.out.println("AU AU AU");
}
}

```java
public class Main {
    no usages
    public static void main(String[] args) {

        Animal aCat = new Cat();
        aCat.speak();
        Animal aDog = new Dog();
        aDog.speak();
        aDog.name = "Pepper";
        aCat.name = "Salty";
        aDog.weight = 25;
        aCat.weight = 7;

        System.out.println(aDog.name);
        System.out.println(aCat.name);
        System.out.println(aCat.weight);
```

```
meowww
AU AU AU
Pepper
Salty
7

Process finished with exit code 0
```

```
/*
   So, they share:
   2 Properties
   1 Method
      - same names/method, but different outputs.


   We can add extra security and organization by forcing to fill
those proprieties once instance created, by using the Method …….
   We could add also some extra security to those proprieties by
using the technic …….
   The fact that method speak() (which is an  …… Method), is
common with other classes but outputs different values, its also
called …… .

*/
```

INTERVIEW QUESTION!

What's the difference between

ABSTRACT

and

INTERFACE ?

ABSTRACT

You can only extend one Class

They can share Properties with different values.

Can have abstract methods and non abstract methods

INTERFACE

You can implement as many interfaces as you would like in a single Class. (Separated by a comma ,)

Its proprieties have to be final and static.

Can only have methods signature

Accessible to all classes

By the way,

Static means _____ .
Final means _____ .

So, **ABSTRACT** classes when you have all **closely related classes**, **same functionality**, **same type of fields** available.

And, **INTERFACE** when you have **UNRELATED classes** that you want to a **certain thing**.

LEAGUE OF LEGENDS FANS

What type of objects, in a game, would be wise to implement **INTERFACE** ?

LET'S CONSIDER ABOUT SHOOTING IT...

Which objects would be vulnerable to **HITS**, which consequently be **destroyed**?

but also, **COMPLETELY DIFFERENT**.

Turrents

Wards

Champions

Drakes

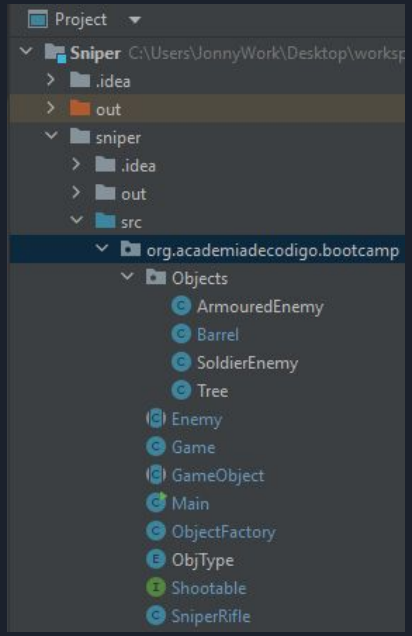
Inhibitors

They are all **UNIQUE**, and **NON-RELATED** objects.
But they share some things in common,

- Health
- Destroyed or !Destroyed

Using **INTERFACE**, we can set this up.

And probably some other ****




```java
package org.academiadecodigo.bootcamp;

no usages  Joao Carneiro *
public class Main {

    no usages  Joao Carneiro *
    public static void main(String[] args) {

        Game newGame = new Game( maxObjects: 10, sniperDmg: 25,  maxShots: 50);

        newGame.start();

    }

}
```

```java
2 usages  Joao Carneiro *
public class Game {

    6 usages
    GameObject[] gameObjects;
    3 usages
    SniperRifle sniperRifle;
    1 usage
    int defaultMaxObj;

    1 usage  Joao Carneiro *
    public Game(int maxObjects,int sniperDmg, int maxShots) {
        defaultMaxObj = maxObjects;
        sniperRifle = new SniperRifle(sniperDmg, maxShots: 50); //  dmg per bullet , max shots per round
    }

    1 usage  Joao Carneiro *
    public void start() {

        gameObjects = createObjects( maxObjects: 10);
        shoot();
    }

    1 usage  Joao Carneiro *
    public GameObject[] createObjects(int maxObjects) {
            return ObjectFactory.createScenario( maxObjs: 10);
    }

    1 usage  Joao Carneiro *
    public void shoot() {
        for (int x = 0; x < gameObjects.length ; x++) {
            if (gameObjects[x] instanceof Shootable) {
                System.out.println(gameObjects[x]);   // we can thank to toString method
                if ((sniperRifle.shootTarget((Shootable)gameObjects[x])) == 0) {   // will return 0 if no more shots available
                    System.out.println("Game over, max shots exceed!");
                    return;                                                 // game is over.
                }
                System.out.println(gameObjects[x].getMessage());
            }
        }
        System.out.println("\n\tSUCCESS!!!!\n\tMission complete. " + sniperRifle.getShotsFired() + " shots fired");
    }
}
```

```java
public class SniperRifle {
    5 usages
    private int bulletDamage;
    2 usages
    private final float HIT_PROB = 0.8f;
    2 usages
    private int shotsFired = 0;
    5 usages
    private int maxShots = 0;


    1 usage  Joao Carneiro *
    SniperRifle(int dmg, int maxShots) {
        this.bulletDamage = dmg;
        this.maxShots = maxShots;
    }
    2 usages  Joao Carneiro *
    public int shootTarget(Shootable target) {  // lets recursion here
        if (maxShots <= 0) {
            return 0;
        }
        if (!target.isDestroyed()) {
            if (Math.random() < HIT_PROB) {
                target.hit(bulletDamage);
                System.out.println("SHOOOOOOT\t" + bulletDamage + " DMG per bullet \t Shots left: " + maxShots);
                shotsFired++;
                maxShots--;
            } else {
                System.out.println("We missed the shot!");
                maxShots--;
            }
            shootTarget(target);
        }
        return 1;
    }
```

```java
public class ObjectFactory {
    1 usage  Joao Carneiro *
    public static GameObject[] createScenario(int maxObjs) {
        int fillOdd = 0;
        int fillOdd2 = 0;

        GameObject[] temp = new GameObject[maxObjs];

        // 60% Change enemies && 40% Chance Trees
        System.out.println("============ Creating Scenario ==========\n");
        System.out.println("** 60% Chance enemies && 40% Chance Trees / Barrels");

        for (int x = 0 ; x < temp.length; x++) {
            fillOdd = (int)(Math.random() * 100);
            if (fillOdd >= 40) {
                fillOdd2 = (int)(Math.random() * 100);
                if (fillOdd2 > 50) {
                    temp[x] = makeNew(ObjType.ENEMYSOLDIER);
                    System.out.println("Adding an Soldier Enemy to the Scenario!");
                } else {
                    temp[x] = makeNew(ObjType.ENEMYARMOURED);
                    System.out.println("Adding an Armoured Enemy to the Scenario!");
                }
            } else {
                fillOdd2 = (int)(Math.random() * 100);
                if (fillOdd2 > 50) {
                    temp[x] = makeNew(ObjType.BARREL);
                    System.out.println("Adding a BARREL to the Scenario!");
                } else {
                    temp[x] = makeNew(ObjType.TREE);
                    System.out.println("Adding a Tree to the Scenario!");
                }
            }
        }
        System.out.println("\n======= SCENARIO CREATED! ========");
        return temp;
    }
    4 usages  Joao Carneiro *
    public static GameObject makeNew(ObjType type) {
        switch (type) {
            case TREE:  return new Tree();
            case ENEMYARMOURED: return new ArmouredEnemy(100);
            case ENEMYSOLDIER: return new SoldierEnemy();
            case BARREL: return new Barrel();
            default: return new Tree();
        }

    }
```

```java
package org.academiadecodigo.bootcamp;

8 usages  4 implementations  Joao Carneiro *
public interface Shootable {

    3 usages  3 implementations  Joao Carneiro
    public abstract void hit(int value);

    2 usages  2 implementations  new *
    public abstract boolean isDestroyed();

}
```

```java
public class Barrel extends GameObject implements Shootable {
    3 usages
    private boolean destroyed = false;


    3 usages  Joao Carneiro *
    public void hit(int dmg) {
        if (!destroyed) {
            setDestroyed();
        }
    }

    1 usage  Joao Carneiro *
    public void setDestroyed() { destroyed = true; }

    2 usages  Joao Carneiro *
    @Override
    public boolean isDestroyed() { return destroyed; }
1 usage  Joao Carneiro
@Override
    public String getMessage() { return getClass().getSimpleName() + " is getting shoot and will be disabled!"; }

    Joao Carneiro *
    @Override
    public String toString() {
        return "BARREL FOUND!! My currently status is: " + ((isDestroyed()) ? "destroyed" : "active!");

    }

}
```

```java
public abstract class Enemy extends GameObject implements Shootable {
    3 usages
    private int health = 100;
    2 usages
    private boolean destroyed = false;

    3 usages  1 override  Joao Carneiro *
    public void hit(int dmg) {
            this.health -= dmg;
            if (this.health <= 0) {
                setDestroyed();
            }
    }

    4 usages  Joao Carneiro
    public int getHealth() { return health; }

    1 usage  Joao Carneiro *
    public void setDestroyed() { destroyed = true; }
    2 usages  Joao Carneiro *
    public boolean isDestroyed() { return destroyed; }

    1 usage  2 overrides  Joao Carneiro *
    @Override
    public String getMessage() { return "Enemy Abstract getMessage method"; }

}
```

Thank you!